

Verified End-to-End Test Results

GitHub Actions + Playwright + Microsoft Entra Certificate-Based Authentication

Customer-shareable test report and reproduction runbook

VERDICT: PASS - 35 OF 35 END-TO-END CHECKS

Document field	Value
Version	1.0
Prepared	September 16, 2026
Runtime proof completed	September 15, 2026
Classification	Customer-shareable; sanitized technical evidence
Runtime-tested commit	b37f4088b8f6e5eebebb11bba91ad798d4c9b72a
Current sanitized main	587571d63ada7ae631db63edd483074bb57f5908

Important: This report contains no password, token, private key, certificate, passphrase, raw sign-in record, tenant ID, subscription ID, user/object ID, policy ID, deployed hostname or IP address. Replace every angle-bracket placeholder only in a secure local operator session.

1. Executive summary	5
2. Proof identity and integrity.....	5
3. Why the proof uses two linked gates	6
3.1 Gate 1 - cloud execution.....	6
3.2 Gate 2 - Conditional Access enforcement.....	6
3.3 Separation-of-duties rationale	6
3.4 Private Endpoint versus Private Link Service	6
4. Enterprise requirement coverage.....	6
5. Complete 35-check result summary	8
6. Detailed result for every test	10
Cross-gate binding.....	10
T01 - Cloud, browser, policy and restoration evidence share one proof set.....	10
GitHub control plane	10
T02 - Exact workflow run completed successfully	10
T03 - Tested branch and commit are exact.....	10
T04 - One exact verification job passed	10
GitHub job execution	11
T05 - Dispatch identifier was validated	11
T06 - Tested revision was checked out.....	11
T07 - Private Key Vault DNS was verified	11
T08 - GitHub OIDC was exchanged and CBA credentials were retrieved.....	11
T09 - Retrieved PFX was validated	12
T10 - POC code type-check passed.....	12
T11 - Headless CBA feasibility test passed.....	12
T12 - Bounded test evidence was validated.....	12
T13 - Bounded evidence artifact was uploaded	13
T14 - Runtime credentials were removed.....	13
Bounded evidence	13
T15 - Identity receipt SHA-256 matched	13
T16 - Network receipt SHA-256 matched.....	13
Playwright identity.....	14
T17 - Exact identity commitment, run and SHA were verified	14
Local browser controls.....	14
T18 - Headed Chromium rendered the exact identity	14
T19 - Five independent headless sessions passed	14
T20 - Wrong-origin certificate isolation passed	14

T21 - Authenticated session reused without a new CBA request	15
Private credential path	15
T22 - Runner resolved and read Key Vault only at the private address	15
GitHub workload identity	15
T23 - OIDC issuer, audience, subject, repository, run and SHA were exact	15
Ephemeral runner identity	16
T24 - ACI runner name, label, subnet and operating system were exact	16
Azure control plane	16
T25 - Azure CLI was on the exact subscription and tenant	16
Azure private network	16
T26 - Key Vault public access remained disabled	16
T27 - Key Vault Private Endpoint was succeeded and approved	16
T28 - Private DNS mapped the vault to the exact private address	17
Azure runner network	17
T29 - Runner subnet remained delegated to ACI and attached to NAT	17
Conditional Access positive	17
T30 - MFA-classified certificate satisfied the exact lab policy	17
Conditional Access negative	17
T31 - SFA-classified certificate failed only the exact lab policy	17
Conditional Access restoration	18
T32 - Lab policy was returned to report-only	18
T33 - Every temporarily isolated managed policy was restored	18
Live cleanup	18
T34 - Ephemeral ACI container no longer existed	18
T35 - Ephemeral GitHub runner was deregistered	19
7. Verify the published proof without tenant mutation	20
Step 1 - Download and hash the public proof	20
Step 2 - Validate the sanitized verdict	20
Step 3 - Inspect the public workflow record	20
Step 4 - Confirm the release points to the tested commit	20
8. Run the complete end-to-end test in your own environment	21
8.1 Time and impact	21
8.2 Prerequisites	21
8.3 Step 0 - Clone and establish a clean operator context	21
8.4 Step 1 - Run static preflight checks	22
8.5 Step 2 - Preview and deploy the Azure boundary	22

8.6 Step 3 - Create the disposable PKI	22
8.7 Step 4 - Deploy the relying application.....	22
8.8 Step 5 - Configure Entra CBA and the report-only lab policy	23
8.9 Step 6 - Run local feasibility controls	23
8.10 Step 7 - Publish runtime assets and configure GitHub OIDC	23
8.11 Step 8 - Run Playwright on the ephemeral Azure runner	23
8.12 Step 9 - Regenerate local controls against the cloud proof set	23
8.13 Step 10 - Run the bounded Conditional Access proof.....	23
8.14 Step 11 - Generate and inspect the final report	24
8.15 Step 12 - Verify safe completion	24
8.16 Step 13 - Recover an interrupted policy transaction	24
8.17 Step 14 - Teardown	24
9. Expected outputs and decision rules	25
10. Troubleshooting guide.....	25
11. Public-template hardening validation	26
12. Security and privacy handling	27
13. Limitations and nonclaims.....	27
14. Operator evidence record for a new run	27
15. Authoritative references	28

1. Executive summary

The solution proved unattended Playwright authentication from GitHub Actions to a real Microsoft Entra-protected application by using a one-job Azure Container Instances runner, GitHub OIDC, private Azure Key Vault retrieval and Microsoft Entra certificate-based authentication.

Acceptance did not rely on a screenshot or browser redirect. Thirty-five independent checks correlated GitHub, Azure, Playwright, certificate, Conditional Access, restoration and cleanup evidence to one deterministic proof set. Every check passed.

Positive control: The certificate carrying the approved policy OID was classified as multiFactorAuthentication and satisfied the exact phishing-resistant lab policy.

Negative control: A second certificate for the same identity omitted the MFA policy OID, was classified as singleFactorAuthentication and failed the exact lab policy with Entra error 500187. That expected rejection is a successful negative test.

Restoration: Three temporarily isolated Microsoft-managed policies were restored to their exact baselines, and the lab policy returned to report-only.

Cleanup: Live Azure and GitHub queries found zero matching ACI containers and zero matching repository runners.

Important: The full tenant-mutating 35/35 runtime proof is anchored to commit b37f4088b8f6e5eebebb11bba91ad798d4c9b72a. The current sanitized public main commit 587571d63ada7ae631db63edd483074bb57f5908 passed static validation, live Azure what-if, proof replay and post-merge CI, but was not represented as a new tenant-mutating 35/35 run. Run the fresh procedure in Section 8 before claiming runtime proof for a newer SHA.

2. Proof identity and integrity

Proof item	Verified value
Repository	samitks77/dev-agents-solutions
Branch	main
Runtime-tested commit	b37f4088b8f6e5eebebb11bba91ad798d4c9b72a
GitHub Actions run	35011709838 (workflow_dispatch)
Proof-set ID	a9f845ff86f347080fcf1609de85c9c69bc7a6afaa3f09fc7fe370914282d40a
Evidence transport	Mandatory private GitHub Actions artifact
Checks	35 passed, 0 failed
Public proof asset SHA-256	70b627da3aa900ec7bb865be2bcc92cbf2bd8334d15eba2aa76b46c1b11c4681
Internal JSON report SHA-256	a8c40023472e1fbb4392e124567365b8e73f1c3f2ae20fb4ea851315b4b68384
Cleanup	ACI deleted; GitHub runner deregistered; live match counts zero

Runtime workflow record: [GitHub Actions run 35011709838](#)

Published sanitized proof: [Entra CBA Playwright verified proof - 2026-09-15](#)

Machine-readable proof asset: [entra-cba-proof-public.json](#)

Current reusable solution: [solutions/entra-cba-playwright](#)

3. Why the proof uses two linked gates

3.1 Gate 1 - cloud execution

GitHub Actions dispatches a uniquely labeled job. A one-job ACI runner registers from a delegated subnet, validates the immutable GitHub OIDC identity, retrieves the PFX and passphrase from a private Key Vault path, runs Playwright and uploads bounded evidence. The launcher then deletes the container and verifies runner deregistration.

3.2 Gate 2 - Conditional Access enforcement

An operator-controlled transaction proves that an MFA-classified certificate succeeds and an SFA-classified certificate fails the exact phishing-resistant policy. Only an explicit allowlist of interfering policies can be temporarily changed, and their original exclusions and invariant hashes are persisted before mutation and restored in finally.

3.3 Separation-of-duties rationale

The Conditional Access transaction requires privileged Microsoft Graph permissions and an exclusive maintenance window. Keeping it outside routine repository CI reduces the impact of a compromised workflow and prevents ordinary source changes from receiving tenant-wide policy authority.

3.4 Private Endpoint versus Private Link Service

A Key Vault Private Endpoint is required because the runner is a client of Key Vault. Azure Private Link Service is not required because the runner exposes no inbound service; it polls GitHub and accesses cloud services over outbound connections. NAT Gateway separately provides predictable outbound egress.

4. Enterprise requirement coverage

Requirement	Verified implementation and evidence	Result
Real Entra authentication from GitHub Actions	Ephemeral ACI runner executes Playwright against the real Entra-protected application.	PASS
Unattended authentication without password storage	Dedicated test user authenticates through Entra CBA; the PFX is retrieved from private Key Vault at runtime.	PASS
No broad MFA exception	MFA-classified certificate satisfies phishing-resistant strength; no tenant-wide bypass is used.	PASS
Prove Conditional Access causality	SFA-classified certificate fails the exact lab policy with error 500187.	PASS
Isolate unrelated tenant-wide policies	Only three explicitly approved managed policies are temporarily isolated, journaled and restored.	PASS

Requirement	Verified implementation and evidence	Result
Fixed runner egress	ACI-delegated subnet uses NAT Gateway with one static public IP and no public IP prefix.	PASS
Private certificate retrieval	Key Vault public access is disabled; Private Endpoint, NIC, DNS record and VNet link agree.	PASS
No long-lived Azure credential in GitHub	GitHub OIDC uses immutable repository identity and exact claim validation before token exchange.	PASS
Prevent wrong-origin certificate disclosure	Playwright scopes the client certificate to the exact Entra certificate-authentication origin and runs a negative control.	PASS
Repeatability and session reuse	Five fresh headless sessions and one authenticated-state reuse scenario pass.	PASS
Bounded proof and cleanup	Sanitized artifact hashes match; credentials are removed; ACI and GitHub runner counts return to zero.	PASS

5. Complete 35-check result summary

The following list is generated from the passing read-only proof replay. Detailed rationale and procedure for every check appear in Section 6.

ID	Stage and test	Result
T01	Cross-gate binding: Cloud, browser, policy and restoration evidence share one proof set	PASS
T02	GitHub control plane: Exact workflow run completed successfully	PASS
T03	GitHub control plane: Tested branch and commit are exact	PASS
T04	GitHub control plane: One exact verification job passed	PASS
T05	GitHub job execution: Dispatch identifier was validated	PASS
T06	GitHub job execution: Tested revision was checked out	PASS
T07	GitHub job execution: Private Key Vault DNS was verified	PASS
T08	GitHub job execution: GitHub OIDC was exchanged and CBA credentials were retrieved	PASS
T09	GitHub job execution: Retrieved PFX was validated	PASS
T10	GitHub job execution: POC code type-check passed	PASS
T11	GitHub job execution: Headless CBA feasibility test passed	PASS
T12	GitHub job execution: Bounded test evidence was validated	PASS
T13	GitHub job execution: Bounded evidence artifact was uploaded	PASS
T14	GitHub job execution: Runtime credentials were removed	PASS
T15	Bounded evidence: Identity receipt SHA-256 matched	PASS
T16	Bounded evidence: Network receipt SHA-256 matched	PASS
T17	Playwright identity: Exact identity commitment, run and SHA were verified	PASS
T18	Local browser controls: Headed Chromium rendered the exact identity	PASS
T19	Local browser controls: Five independent headless sessions passed	PASS
T20	Local browser controls: Wrong-origin certificate isolation passed	PASS
T21	Local browser controls: Authenticated session reused without a new CBA request	PASS
T22	Private credential path: Runner resolved and read Key Vault only at the private address	PASS
T23	GitHub workload identity: OIDC issuer, audience, subject, repository, run and SHA were exact	PASS
T24	Ephemeral runner identity: ACI runner name, label, subnet and operating system were exact	PASS
T25	Azure control plane: Azure CLI was on the exact subscription and tenant	PASS

ID	Stage and test	Result
T26	Azure private network: Key Vault public access remained disabled	PASS
T27	Azure private network: Key Vault Private Endpoint was succeeded and approved	PASS
T28	Azure private network: Private DNS mapped the vault to the exact private address	PASS
T29	Azure runner network: Runner subnet remained delegated to ACI and attached to NAT	PASS
T30	Conditional Access positive: MFA-classified certificate satisfied the exact lab policy	PASS
T31	Conditional Access negative: SFA-classified certificate failed only the exact lab policy	PASS
T32	Conditional Access restoration: Lab policy was returned to report-only	PASS
T33	Conditional Access restoration: Every temporarily isolated managed policy was restored	PASS
T34	Live cleanup: Ephemeral ACI container no longer existed	PASS
T35	Live cleanup: Ephemeral GitHub runner was deregistered	PASS

6. Detailed result for every test

Cross-gate binding

T01 - Cloud, browser, policy and restoration evidence share one proof set

Why this test exists: Prevents evidence from different runs, commits, identities or policy transactions from being combined into a false pass.

How it was tested: Recompute the deterministic SHA-256 proof-set identifier from the cloud-run facts and compare it with every browser, Conditional Access and restoration receipt.

Acceptance rule: Every required receipt contains the same non-empty proof-set identifier and exact tested source revision.

Observed result: PASS - All cloud, local browser, policy and restoration records matched one proof set.

Evidence source: Deterministic SHA-256 proof-set binding

GitHub control plane

T02 - Exact workflow run completed successfully

Why this test exists: A local receipt is insufficient unless GitHub independently reports that the intended workflow run completed successfully.

How it was tested: Query the live GitHub Actions API for the recorded run and compare repository, workflow, event, status and conclusion.

Acceptance rule: The exact workflow-dispatch run is completed with a successful conclusion.

Observed result: PASS - Workflow run 35011709838 completed successfully.

Evidence source: Live GitHub API

T03 - Tested branch and commit are exact

Why this test exists: Ensures the evidence describes the reviewed source and not a mutable or different revision.

How it was tested: Compare the live run head branch and SHA with the immutable values recorded by the launcher and proof set.

Acceptance rule: Branch is main and head SHA equals the runtime-tested commit.

Observed result: PASS - The run was bound to main at b37f4088b8f6e5eebebb11bba91ad798d4c9b72a.

Evidence source: Live GitHub API

T04 - One exact verification job passed

Why this test exists: Rejects ambiguous evidence from duplicate, retried or unrelated jobs.

How it was tested: Filter the live run jobs by the unique verification label and expected job identity.

Acceptance rule: Exactly one matching verification job exists and it succeeded.

Observed result: PASS - One matching job was found and passed.

Evidence source: Live GitHub API

GitHub job execution

T05 - Dispatch identifier was validated

Why this test exists: Binds the queued workflow to the one-time runner launch and prevents another queued job from using the runner.

How it was tested: Validate the operator-supplied verification identifier before any credential access.

Acceptance rule: The workflow input equals the runner's unique verification identifier.

Observed result: PASS - The dispatch identifier validation step passed.

Evidence source: Live GitHub API

T06 - Tested revision was checked out

Why this test exists: Guarantees the job executed the exact source revision used by the proof.

How it was tested: Use the immutable Actions checkout and verify the checked-out SHA against the expected run head.

Acceptance rule: Checked-out HEAD equals the recorded tested commit.

Observed result: PASS - The checkout step passed at the exact runtime-tested SHA.

Evidence source: Live GitHub API

T07 - Private Key Vault DNS was verified

Why this test exists: Prevents certificate retrieval over a public or unexpected endpoint.

How it was tested: Resolve the standard vault hostname inside the runner and compare the answer with the approved Private Endpoint address from ARM.

Acceptance rule: One private address is returned and it equals the Private Endpoint NIC and private DNS record.

Observed result: PASS - Private DNS verification passed.

Evidence source: Live GitHub API

T08 - GitHub OIDC was exchanged and CBA credentials were retrieved

Why this test exists: Proves passwordless workload federation and runtime-only certificate retrieval.

How it was tested: Validate OIDC claims, exchange the token with Azure, and read the separate PFX and passphrase secrets from private Key Vault.

Acceptance rule: OIDC claims are exact; Azure token exchange and both secret reads succeed without a stored GitHub client secret.

Observed result: PASS - OIDC exchange and private credential retrieval passed.

Evidence source: Live GitHub API

T09 - Retrieved PFX was validated

Why this test exists: Detects a wrong, corrupt, expired or mismatched certificate before browser execution.

How it was tested: Open the downloaded PFX, validate its expected certificate properties and match it to the dedicated test identity.

Acceptance rule: The PFX is readable only with the runtime passphrase and matches the expected test certificate.

Observed result: PASS - PFX validation passed.

Evidence source: Live GitHub API

T10 - POC code type-check passed

Why this test exists: Prevents execution of a test bundle with TypeScript contract errors.

How it was tested: Run the pinned TypeScript compiler with no output before browser execution.

Acceptance rule: The compiler returns exit code zero.

Observed result: PASS - Type-check passed.

Evidence source: Live GitHub API

T11 - Headless CBA feasibility test passed

Why this test exists: Demonstrates unattended browser authentication from the ephemeral Azure runner.

How it was tested: Launch Chromium through Playwright with the client certificate scoped only to the Entra certificate-authentication origin, then assert exact application identity claims.

Acceptance rule: The browser reaches the relying application and the expected identity commitment is produced.

Observed result: PASS - Headless CBA test passed.

Evidence source: Live GitHub API

T12 - Bounded test evidence was validated

Why this test exists: Stops raw identities, tokens, traces or oversized diagnostic data from becoming public workflow output.

How it was tested: Validate the allowlisted evidence schema, required fields, file count and size before publication.

Acceptance rule: Only the bounded identity and network receipts are accepted.

Observed result: PASS - Evidence validation passed.

Evidence source: Live GitHub API

T13 - Bounded evidence artifact was uploaded

Why this test exists: Makes the proof retrievable without falling back to public job-log disclosure.

How it was tested: Upload the validated receipts as the mandatory GitHub Actions artifact.

Acceptance rule: Artifact upload succeeds; absence or quota failure fails the workflow.

Observed result: PASS - Mandatory artifact upload passed.

Evidence source: Live GitHub API

T14 - Runtime credentials were removed

Why this test exists: Reduces exposure after the one-job runner finishes.

How it was tested: Delete PFX and passphrase material in the always-run cleanup step.

Acceptance rule: Credential-removal step succeeds even when an earlier test step fails.

Observed result: PASS - Runtime credential removal passed.

Evidence source: Live GitHub API

Bounded evidence

T15 - Identity receipt SHA-256 matched

Why this test exists: Detects any modification between workflow production, artifact transport and local replay.

How it was tested: Re-hash the downloaded identity receipt and compare it with the post-cleanup launcher journal.

Acceptance rule: The computed and recorded SHA-256 values are identical.

Observed result: PASS - Identity receipt integrity matched.

Evidence source: Local cryptographic replay

T16 - Network receipt SHA-256 matched

Why this test exists: Protects the network and OIDC facts from post-run alteration.

How it was tested: Re-hash the downloaded network receipt and compare it with the launcher journal.

Acceptance rule: The computed and recorded SHA-256 values are identical.

Observed result: PASS - Network receipt integrity matched.

Evidence source: Local cryptographic replay

Playwright identity

T17 - Exact identity commitment, run and SHA were verified

Why this test exists: A page-load success alone does not prove which user, tenant, application or source revision authenticated.

How it was tested: Compare the sanitized identity commitment and exact run/SHA fields with ignored operator state and the live GitHub run.

Acceptance rule: All identity commitment and run-binding values agree.

Observed result: PASS - Exact identity commitment and source binding passed.

Evidence source: Re-hashed Playwright receipt

Local browser controls

T18 - Headed Chromium rendered the exact identity

Why this test exists: Provides a human-observable check in addition to machine assertions.

How it was tested: Run the same certificate flow in headed Chromium and assert the relying application's identity fields.

Acceptance rule: The visible claims match the dedicated test identity and the bounded receipt passes.

Observed result: PASS - Headed browser control passed.

Evidence source: Bounded local Playwright receipt

T19 - Five independent headless sessions passed

Why this test exists: Distinguishes a repeatable authentication path from a one-off cached or timing-dependent success.

How it was tested: Create five fresh browser contexts and execute the complete CBA sign-in independently.

Acceptance rule: All five runs pass without shared authenticated state.

Observed result: PASS - Five of five fresh sessions passed.

Evidence source: Bounded local Playwright receipt

T20 - Wrong-origin certificate isolation passed

Why this test exists: Proves the private key is not offered to an origin outside the approved Entra certificate-authentication endpoint.

How it was tested: Navigate a test context to a non-approved origin while the certificate configuration remains exact-origin scoped.

Acceptance rule: The non-approved origin does not receive a client certificate.

Observed result: PASS - Wrong-origin negative control passed.

Evidence source: Bounded local Playwright receipt

T21 - Authenticated session reused without a new CBA request

Why this test exists: Validates deterministic storage-state reuse for subsequent tests without repeatedly exposing certificate material.

How it was tested: Create an authenticated state, start a new context from that state and observe whether the certificate-authentication origin is requested again.

Acceptance rule: The relying application opens successfully without a new CBA request.

Observed result: PASS - Session reuse passed.

Evidence source: Bounded local Playwright receipt

Private credential path

T22 - Runner resolved and read Key Vault only at the private address

Why this test exists: Demonstrates that runtime secrets are not retrieved through Key Vault public networking.

How it was tested: Correlate in-run DNS and secret-read facts with the exact Private Endpoint and live ARM topology.

Acceptance rule: Vault public access is disabled, DNS resolves to the expected private address and both reads succeed.

Observed result: PASS - Private credential path passed.

Evidence source: Re-hashed runner receipt

GitHub workload identity

T23 - OIDC issuer, audience, subject, repository, run and SHA were exact

Why this test exists: Prevents a token from another repository, owner, environment, run or source revision from being accepted.

How it was tested: Decode the GitHub OIDC token before exchange and compare every bounded claim with live repository metadata and the immutable subject.

Acceptance rule: All expected claims match exactly before Azure token exchange.

Observed result: PASS - Immutable workload identity validation passed.

Evidence source: Re-hashed runner receipt

Ephemeral runner identity

T24 - ACI runner name, label, subnet and operating system were exact

Why this test exists: Confirms that the job ran on the intended one-time Azure compute rather than another self-hosted runner.

How it was tested: Compare workflow runner facts with the launch journal and verified live subnet contract.

Acceptance rule: Name, unique label, Linux OS and private address within the recorded runner CIDR all match.

Observed result: PASS - Ephemeral runner identity passed.

Evidence source: Re-hashed runner receipt

Azure control plane

T25 - Azure CLI was on the exact subscription and tenant

Why this test exists: Avoids collecting or mutating evidence in the wrong Azure or Entra boundary.

How it was tested: Re-query the current Azure account and compare its identifiers with ignored deployment state.

Acceptance rule: Both subscription and tenant identifiers match the recorded lab.

Observed result: PASS - Azure context validation passed.

Evidence source: Live Azure API

Azure private network

T26 - Key Vault public access remained disabled

Why this test exists: Confirms the private secret path remained enforced after testing.

How it was tested: Read the live Key Vault network configuration from ARM.

Acceptance rule: Public network access is disabled and the default network action is Deny.

Observed result: PASS - Key Vault remained private-only.

Evidence source: Live Azure API

T27 - Key Vault Private Endpoint was succeeded and approved

Why this test exists: Proves the private endpoint is usable and connected to the exact vault resource.

How it was tested: Read the live Private Endpoint, connection state, group ID and NIC from ARM.

Acceptance rule: Exactly one approved vault connection and one NIC are present.

Observed result: PASS - Private Endpoint verification passed.

Evidence source: Live Azure API

T28 - Private DNS mapped the vault to the exact private address

Why this test exists: A Private Endpoint without correct DNS can still cause public resolution or failed secret access.

How it was tested: Compare the private DNS A record, Private Endpoint NIC address and linked VNet.

Acceptance rule: One A record equals the NIC address and one completed non-registering VNet link exists.

Observed result: PASS - Private DNS verification passed.

Evidence source: Live Azure API

Azure runner network

T29 - Runner subnet remained delegated to ACI and attached to NAT

Why this test exists: Provides deterministic outbound egress and prevents an unexpected route table from changing the path.

How it was tested: Read the live subnet, delegation, NAT Gateway, public IP association and route-table state.

Acceptance rule: One ACI delegation, the expected NAT Gateway, one static public IP, no public IP prefix and no UDR are present.

Observed result: PASS - Runner network contract passed.

Evidence source: Live Azure API

Conditional Access positive

T30 - MFA-classified certificate satisfied the exact lab policy

Why this test exists: Proves strong CBA can satisfy phishing-resistant Conditional Access without a broad MFA bypass.

How it was tested: Use the certificate containing the organization-controlled policy OID and correlate the browser result with Entra sign-in evidence.

Acceptance rule: Terminal error is zero, the exact lab policy is success, CBA level is multiFactorAuthentication, the X.509 step succeeds and the modern PKI store is used.

Observed result: PASS - Positive Conditional Access scenario passed.

Evidence source: Sanitized Entra sign-in receipt

Conditional Access negative

T31 - SFA-classified certificate failed only the exact lab policy

Why this test exists: A successful positive path alone cannot prove the authentication-strength policy caused the outcome.

How it was tested: Use a certificate for the same user without the MFA policy OID while temporarily isolating only the approved interfering policies.

Acceptance rule: The exact lab policy fails with error 500187, CBA is singleFactorAuthentication, the X.509 step fails and unrelated isolated policies are notApplied.

Observed result: PASS - The intended negative scenario failed exactly as designed and was counted as PASS.

Evidence source: Sanitized Entra sign-in receipt

Conditional Access restoration

T32 - Lab policy was returned to report-only

Why this test exists: Prevents the temporary enforcement test from remaining active after evidence collection.

How it was tested: Restore the policy before long sign-in-log polling and verify its final state from the transaction receipt.

Acceptance rule: Final state is enabledForReportingButNotEnforced.

Observed result: PASS - Lab policy restoration passed.

Evidence source: Post-transaction read-back receipt

T33 - Every temporarily isolated managed policy was restored

Why this test exists: Ensures the causality test does not leave tenant-wide policy exclusions changed.

How it was tested: Persist baseline exclusion sets and invariant hashes before mutation, then restore and verify every policy in finally.

Acceptance rule: All three approved policies match their exact baselines and the schema-v3 journal reports restored.

Observed result: PASS - Three of three managed policies were restored.

Evidence source: Atomic isolation journal

Live cleanup

T34 - Ephemeral ACI container no longer existed

Why this test exists: Verifies one-job compute destruction and prevents residual runner access.

How it was tested: Query the live Azure API for the run-scoped container name after launcher cleanup.

Acceptance rule: Zero matching ACI container groups exist.

Observed result: PASS - Live ACI match count was zero.

Evidence source: Live Azure API

T35 - Ephemeral GitHub runner was deregistered

Why this test exists: Prevents an offline or reusable runner registration from remaining trusted by the repository.

How it was tested: Query the live repository runner API for the run-scoped name after cleanup.

Acceptance rule: Zero matching repository runners exist.

Observed result: PASS - Live GitHub runner match count was zero.

Evidence source: Live GitHub API

7. Verify the published proof without tenant mutation

This path is safe for a customer review because it downloads only the deliberately sanitized proof summary. It does not require access to the original tenant, subscription, certificate or ignored operator state.

Step 1 - Download and hash the public proof

```
$assetUrl = 'https://github.com/samitks77/dev-agents-solutions/releases/download/entra-cba-playwright-proof-2026-09-15/entra-cba-proof-public.json'
$proofPath = Join-Path $PWD 'entra-cba-proof-public.json'
$expectedSha256 = '70b627da3aa900ec7bb865be2bcc92cbf2bd8334d15eba2aa76b46c1b11c4681'

Invoke-WebRequest -Uri $assetUrl -OutFile $proofPath
$actualSha256 = (Get-FileHash -LiteralPath $proofPath -Algorithm SHA256).Hash.ToLowerInvariant()
if ($actualSha256 -ne $expectedSha256) {
    throw 'Published proof SHA-256 does not match.'
}
```

Step 2 - Validate the sanitized verdict

```
$proof = Get-Content -LiteralPath $proofPath -Raw | ConvertFrom-Json
if ($proof.verdict -ne 'PASS' -or
    $proof.verification.checksPassed -ne 35 -or
    $proof.verification.checksFailed -ne 0 -or
    -not $proof.cleanup.aciContainerDeleted -or
    -not $proof.cleanup.githubRunnerDeregistered -or
    $proof.privacy.containsCredentials -or
    $proof.privacy.containsPrivateKey -or
    $proof.privacy.containsRawSignInRecords -or
    $proof.privacy.containsTenantUserObjectOrSubscriptionIds) {
    throw 'Published proof did not satisfy the customer-safe acceptance checks.'
}

$proof | Select-Object verdict, scope, source, verifiedControls, conditionalAccess, cleanup, verification,
privacy
```

Step 3 - Inspect the public workflow record

```
gh run view 35011709838 `
  --repo samitks77/dev-agents-solutions `
  --json status,conclusion,event,headBranch,headSha,url,jobs
```

Expected result: status completed, conclusion success, event workflow_dispatch, branch main and head SHA equal to the runtime-tested commit listed in Section 2.

Step 4 - Confirm the release points to the tested commit

```
gh release view entra-cba-playwright-proof-2026-09-15 `
  --repo samitks77/dev-agents-solutions `
  --json tagName,targetCommitish,url,assets
```

Expected result: targetCommitish equals b37f4088b8f6e5eebebb11bba91ad798d4c9b72a and the release contains entra-cba-proof-public.json.

8. Run the complete end-to-end test in your own environment

Important: Use a dedicated non-production identity and an isolated lab application. The Conditional Access phase changes policy state temporarily and requires an exclusive maintenance window. Do not run it casually in a production tenant.

8.1 Time and impact

Mode	Typical duration	Tenant or cloud change
Public proof verification	5 minutes	None
Existing-operator evidence replay	5-10 minutes	None; live read-only GitHub/Azure queries
Fresh cloud-runner gate	5-10 minutes	Temporary ACI and runner registration
Fresh Conditional Access gate	At least 57 minutes	Bounded policy enforcement, isolation and restoration

8.2 Prerequisites

- PowerShell 7.
- Node.js 22 or later and npm.
- Azure CLI authenticated to the intended subscription and tenant.
- GitHub CLI authenticated to the intended repository with ADMIN permission.
- Microsoft Graph PowerShell Authentication 2.39.0.
- OpenSSL 3 and Playwright Chromium.
- A dedicated disposable Entra test user; never use a production identity.
- Azure permissions to deploy the lab resources and Microsoft Graph consent for the scopes explicitly checked by the scripts.
- An organization-controlled certificate policy OID.
- Three non-overlapping RFC 1918 ranges: one VNet and two contained subnets, each subnet with at least eight addresses.

8.3 Step 0 - Clone and establish a clean operator context

```
git clone https://github.com/<github-owner>/<repository-name>.git
Set-Location .\<repository-name>\solutions\entra-cba-playwright
git switch main
git pull --ff-only
git status --short --branch

$repository = '<github-owner>/<repository-name>'
$subscription = '<azure-subscription-id-or-name>'
$tenantId = '<entra-tenant-id>'
$resourceGroup = '<resource-group-name>'
$location = '<azure-region>'
$testUserUpn = '<dedicated-test-user-upn>'
$certificatePolicyOid = '<organization-controlled-certificate-policy-oid>'
$virtualNetworkAddressPrefix = '<virtual-network-cidr>'
$runnerSubnetAddressPrefix = '<aci-runner-subnet-cidr>'
$privateEndpointSubnetAddressPrefix = '<private-endpoint-subnet-cidr>'

npm ci --ignore-scripts
npx playwright install chromium
az login --tenant $tenantId
az account set --subscription $subscription
az account show
```

```
gh auth status
```

Stop if GitHub or Azure resolves to an unintended identity, repository, tenant or subscription. Keep populated values only in the current secure shell or ignored runtime state.

8.4 Step 1 - Run static preflight checks

```
.\scripts\Test-PublicTemplate.ps1
npm run typecheck
npm run build:app
az bicep build --file .\infra\main.bicep --outfile (Join-Path $env:TEMP 'entra-cba-main.json')

$failed = $false
Get-ChildItem .\scripts -Filter *.ps1 | ForEach-Object {
    $errors = $null
    [void][Management.Automation.Language.Parser]::ParseFile(
        $_.FullName,
        [ref]$null,
        [ref]$errors
    )
    if (@($errors).Count -ne 0) {
        $failed = $true
        $errors | ForEach-Object { Write-Error $_.Message }
    }
}
if ($failed) { throw 'PowerShell parsing failed.' }
```

Expected result: public-template validation passes, TypeScript passes, the application builds, Bicep compiles and every PowerShell script parses.

8.5 Step 2 - Preview and deploy the Azure boundary

```
.\scripts\Deploy-Infrastructure.ps1 `
-Subscription $subscription `
-ExpectedTenantId $tenantId `
-ResourceGroup $resourceGroup `
-Location $location `
-VirtualNetworkAddressPrefix $virtualNetworkAddressPrefix `
-RunnerSubnetAddressPrefix $runnerSubnetAddressPrefix `
-PrivateEndpointSubnetAddressPrefix $privateEndpointSubnetAddressPrefix `
-WhatIf

.\scripts\Deploy-Infrastructure.ps1 `
-Subscription $subscription `
-ExpectedTenantId $tenantId `
-ResourceGroup $resourceGroup `
-Location $location `
-VirtualNetworkAddressPrefix $virtualNetworkAddressPrefix `
-RunnerSubnetAddressPrefix $runnerSubnetAddressPrefix `
-PrivateEndpointSubnetAddressPrefix $privateEndpointSubnetAddressPrefix
```

Review the what-if output before deployment. The script rejects non-canonical, public, overlapping or out-of-VNet CIDRs before Azure mutation.

8.6 Step 3 - Create the disposable PKI

```
.\scripts\New-LabPki.ps1 `
-TestUserUpn $testUserUpn `
-PolicyOid $certificatePolicyOid
```

Expected result: one disposable root CA plus MFA- and SFA-classified user certificates are written only to ignored local state. Never commit the PFX, private key or passphrase.

8.7 Step 4 - Deploy the relying application

```
npm run build:app
.\scripts\Deploy-TestApp.ps1 -TestUsername $testUserUpn
```

Expected result: a single-tenant SPA registration uses only the exact Static Web App redirect URI and renders the minimum identity claims required by the test.

8.8 Step 5 - Configure Entra CBA and the report-only lab policy

```
.\scripts\Connect-EntraCbaLabGraph.ps1 -TenantId $tenantId
.\scripts\Configure-EntraCba.ps1 `
  -TenantId $tenantId `
  -TestUserUpn $testUserUpn
.\scripts\Configure-ConditionalAccess.ps1 `
  -TenantId $tenantId `
  -State enabledForReportingButNotEnforced
```

Read every consent prompt. The configuration scripts snapshot the existing X.509 method and PKI settings, create or validate the dedicated user/group, upload only the disposable CA and keep the lab Conditional Access policy report-only.

8.9 Step 6 - Run local feasibility controls

```
.\scripts\Invoke-LocalFeasibility.ps1 -Headed -ShowProof
.\scripts\Invoke-LocalFeasibility.ps1 -WrongOrigin
.\scripts\Invoke-LocalFeasibility.ps1 -Repeat 5
.\scripts\Invoke-LocalFeasibility.ps1 -ReuseSession
```

These are prechecks. Final acceptance receipts must be regenerated after the cloud run creates the deterministic proof-set ID.

8.10 Step 7 - Publish runtime assets and configure GitHub OIDC

```
.\scripts\Publish-LabAssets.ps1
.\scripts\Configure-GitHubOidc.ps1 `
  -Repository $repository `
  -AllowedBranches @('main')
```

Expected result: the digest-pinned publisher receives secret-write access only for publication; the GitHub workload identity receives secret-read access only. No certificate is stored as a GitHub secret.

8.11 Step 8 - Run Playwright on the ephemeral Azure runner

```
.\scripts\Start-EphemeralGitHubRunner.ps1 `
  -Repository $repository `
  -Dispatch `
  -Ref main
```

Do not manually dispatch the workflow first. The launcher binds one queued run to one unique runner label, validates the exact pushed SHA, waits for completion, downloads the mandatory artifact and performs cleanup in finally.

8.12 Step 9 - Regenerate local controls against the cloud proof set

```
$runnerProof = Get-Content .\lab-state\runner.json -Raw | ConvertFrom-Json
$proofSetId = $runnerProof.proofSetId

.\scripts\Invoke-LocalFeasibility.ps1 -Headed -ProofSetId $proofSetId
.\scripts\Invoke-LocalFeasibility.ps1 -WrongOrigin -ProofSetId $proofSetId
.\scripts\Invoke-LocalFeasibility.ps1 -Repeat 5 -ProofSetId $proofSetId
.\scripts\Invoke-LocalFeasibility.ps1 -ReuseSession -ProofSetId $proofSetId
```

8.13 Step 10 - Run the bounded Conditional Access proof

Important: Schedule an exclusive maintenance window. Supply only policy IDs reviewed for the target tenant. The documented timings require at least 57 minutes of Graph token lifetime before mutation.

```
$policyIds = @(
  '<managed-policy-id-1>',
  '<managed-policy-id-2>',
  '<managed-policy-id-3>'
) -join ','

.\scripts\Invoke-ConditionalAccessProof.ps1 `
  -TenantId $tenantId `
  -BrowserScenario Both `
  -InterferingPolicyIdsCsv $policyIds `
  -ConfirmExclusiveConditionalAccessWindow `
```

```
-PropagationSeconds 900 `
-NegativeFinalizationSeconds 120 `
-EvidenceTimeoutMinutes 30
```

Expected result: the MFA certificate succeeds; the SFA certificate receives error 500187 from the exact lab policy; all three interfering policies are notApplied during the negative test; every policy is restored.

8.14 Step 11 - Generate and inspect the final report

```
.\scripts\Show-E2eProof.ps1
Start-Process .\artifacts\showcase\e2e-proof.html
```

The command must report E2E_SHOWCASE_PASS with 35 checks passed and zero failed. Review the report in order: GitHub control plane, job steps, receipt hashes, identity, network, Conditional Access, restoration and cleanup.

8.15 Step 12 - Verify safe completion

- The lab Conditional Access policy is report-only.
- Every managed policy has its exact original application exclusions.
- The isolation journal records status restored.
- No matching ACI container remains.
- No matching GitHub runner remains.
- No PFX, private key, passphrase, token or raw sign-in record was committed or published.

8.16 Step 13 - Recover an interrupted policy transaction

```
.\scripts\Invoke-ConditionalAccessProof.ps1 `
-TenantId $tenantId `
-InterferingPolicyIdsCsv $policyIds `
-ConfirmExclusiveConditionalAccessWindow `
-RestoreIsolationOnly
```

8.17 Step 14 - Teardown

```
.\scripts\Remove-EntraCbaLab.ps1 -TenantId $tenantId
```

The teardown command requires confirmation and removes only recorded lab objects after restoring the recorded X.509 authentication-method baseline and Conditional Access state.

9. Expected outputs and decision rules

Checkpoint	Expected output	Stop condition
Template preflight	PUBLIC_TEMPLATE_PASS	Any tenant value, UPN, resource ID, IP/CIDR, sensitive file or non-public GUID is detected.
Infrastructure preview	What-if succeeds	Unexpected deletion, wrong subscription/tenant, invalid or overlapping network range.
Local browser	Headed, wrong-origin, five fresh sessions and reuse all pass	Identity mismatch, certificate offered to wrong origin, unstable fresh sessions or new CBA request during reuse.
Cloud runner	Workflow success plus mandatory artifact	Wrong SHA/branch, OIDC mismatch, public vault resolution, PFX failure, missing artifact or failed cleanup.
Positive CA	Error 0; exact policy success; MFA CBA	Any failure/reportOnlyFailure for the exact policy or missing modern X.509 evidence.
Negative CA	Error 500187; exact policy failure; SFA CBA	Another policy is causal, the lab policy is report-only/notApplied, or the correlation is ambiguous.
Restoration	Lab report-only; three managed baselines restored	Any exclusion or invariant differs from the baseline.
Final verifier	E2E_SHOWCASE_PASS checks=35 passed=35 failed=0	Any one of the 35 checks fails.

10. Troubleshooting guide

Symptom	Likely cause	Required response
Infrastructure fails before deployment	CIDR is non-canonical, public, outside the VNet, overlapping or too small.	Choose three canonical, non-overlapping RFC 1918 ranges and rerun what-if.
Runner never accepts the job	Workflow is not on the default branch, environment branch allowlist differs, or registration exceeded the bounded window.	Verify main contains the workflow, rerun OIDC configuration and start a new uniquely labeled runner.
Vault resolves publicly or not at all	Private DNS link, A record or Private Endpoint is inconsistent.	Stop. Repair the Private Endpoint/DNS topology; do not bypass the private-path check.
PFX validation fails	Wrong secret version, passphrase, certificate or expired material.	Republish the exact generated assets; never weaken validation.

Symptom	Likely cause	Required response
Positive CBA does not satisfy strength	Policy OID mapping, user binding, CA trust, CRL or propagation is incomplete.	Check the CBA configuration and wait for documented propagation before rerunning.
Negative test fails under another policy	A tenant-wide MFA policy remains causal.	Use only the reviewed isolation allowlist in an exclusive maintenance window, then rerun.
Artifact is missing	Artifact storage is unavailable, quota is exhausted or upload failed.	Treat the workflow as failed. Do not move receipts into public logs.
Policy restoration is uncertain	The transaction was interrupted or concurrent edits were detected.	Run RestoreIsolationOnly and review the schema-v3 journal before any new proof.
Verifier reports a SHA or proof-set mismatch	Evidence comes from different revisions or runs.	Discard the mixed evidence and regenerate every receipt against one pushed commit.

11. Public-template hardening validation

After the runtime proof, the public template was sanitized and merged through pull request 7. This hardening did not alter the historical proof claim; it made future deployments require explicit operator inputs and added CI enforcement.

Validation	Result
Public-template guard	PASS across 50 candidate files
PowerShell parsing	PASS across 23 scripts
CIDR helper tests	PASS: 3 valid topologies, 7 invalid topologies, 4 membership cases
Node dependency audit	PASS: 0 vulnerabilities at validation time
TypeScript and application build	PASS
Bicep compilation	PASS
Tracked Markdown links	PASS across 3 files
Live Azure what-if using ignored lab state	PASS
Read-only proof replay after hardening	PASS: 35/35
Gitleaks v8.30.1 full reachable history	PASS: no leaks
Post-merge GitHub solution-quality workflow	PASS
Post-merge GitHub secret-scan workflow	PASS
Public remote-tree audit	PASS at 587571d63ada7ae631db63edd483074bb57f5908

Important: GitHub Advanced Security's content-scanning API was not available for this repository. The repository's independent gitleaks full-history workflow and local gitleaks v8.30.1 scan both passed. The public-template guard also fails closed on sensitive paths and deployment-specific literals.

12. Security and privacy handling

- No credential, access token, client secret, private key, certificate or PFX passphrase appears in this report.
- No tenant, subscription, user, object, application, group or policy identifier appears in this report.
- No deployed hostname, resource ID, public IP, private IP or network CIDR appears in this report.
- Runtime state remains under Git-ignored .auth, .artifacts, .lab-secrets and .lab-state directories.
- Public evidence contains commitments, status and bounded metadata rather than raw identity or sign-in payloads.
- Fixed GUIDs retained in source are documented public Microsoft Graph, managed-policy-template or Azure built-in-role identifiers.
- Fixed SHA-256 values retained in source are integrity pins, not credentials.

13. Limitations and nonclaims

- This is an experimentally verified integration, not an official Microsoft or GitHub combined reference architecture.
- The proof demonstrates one isolated lab identity and relying application; it does not authorize production rollout without an organization's normal security review.
- The validated compute implementation is an ephemeral ACI self-hosted runner. GitHub-hosted larger runners with Azure private networking remain a separate production option subject to plan and regional availability.
- Private Link Service is intentionally absent because the runner exposes no inbound service.
- The published sanitized proof excludes raw sign-in data and deployment identifiers; an authorized operator retains the private evidence required for live replay.
- Any source change after the runtime-tested commit requires a fresh full run before claiming the new SHA has the same 35/35 runtime proof.

14. Operator evidence record for a new run

Complete this section after a fresh run. Keep tenant and subscription identifiers in approved private records rather than the customer-shareable copy.

Field	Operator entry
Test date/time (UTC)	<date-time>
Operator	<operator-name>
Repository	<github-owner>/<repository-name>
Tested commit SHA	<40-character-sha>
Workflow run ID	<workflow-run-id>

Field	Operator entry
Proof-set ID	<proof-set-sha256>
Final verifier	E2E_SHOWCASE_<PASS-or-FAIL>
Checks	<passed>/<total>
Positive CBA	<PASS-or-FAIL>
Negative CBA	<PASS-or-FAIL>
Policy restoration	<PASS-or-FAIL>
ACI cleanup	<PASS-or-FAIL>
GitHub runner cleanup	<PASS-or-FAIL>
Sanitized report SHA-256	<sha256>
Reviewer	<reviewer-name>

15. Authoritative references

[Playwright client certificates](https://playwright.dev/docs/api/class-browser#browser-new-context-option-client-certificates) - <https://playwright.dev/docs/api/class-browser#browser-new-context-option-client-certificates>

[Microsoft Entra CBA overview](https://learn.microsoft.com/en-us/entra/identity/authentication/concept-certificate-based-authentication) - <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-certificate-based-authentication>

[Configure Entra CBA binding rules](https://learn.microsoft.com/en-us/entra/identity/authentication/how-to-certificate-based-authentication#step-3-configure-an-authentication-binding-policy) - <https://learn.microsoft.com/en-us/entra/identity/authentication/how-to-certificate-based-authentication#step-3-configure-an-authentication-binding-policy>

[Conditional Access authentication strengths](https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-strengths) - <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-strengths>

[GitHub OIDC with Azure](https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-azure) - <https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-azure>

[GitHub ephemeral self-hosted runners](https://docs.github.com/en/actions/reference/runners/self-hosted-runners#ephemeral-runners-for-autoscaling) - <https://docs.github.com/en/actions/reference/runners/self-hosted-runners#ephemeral-runners-for-autoscaling>

[GitHub self-hosted runner hardening](https://docs.github.com/en/actions/reference/security/secure-use#hardening-for-self-hosted-runners) - <https://docs.github.com/en/actions/reference/security/secure-use#hardening-for-self-hosted-runners>

[ACI in a VNet](https://learn.microsoft.com/en-us/azure/container-instances/container-instances-vnet) - <https://learn.microsoft.com/en-us/azure/container-instances/container-instances-vnet>

[ACI with NAT Gateway](https://learn.microsoft.com/en-us/azure/container-instances/container-instances-nat-gateway) - <https://learn.microsoft.com/en-us/azure/container-instances/container-instances-nat-gateway>

[Key Vault Private Link](https://learn.microsoft.com/en-us/azure/key-vault/general/private-link-service) - <https://learn.microsoft.com/en-us/azure/key-vault/general/private-link-service>

[Private Endpoint DNS](https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-dns) - <https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-dns>

[Azure Private Link Service](https://learn.microsoft.com/en-us/azure/private-link/private-link-service-overview) - <https://learn.microsoft.com/en-us/azure/private-link/private-link-service-overview>